03 — Benchmark and Frontend Demo

What you'll learn: how to prove the GP helps (synthetic dropout benchmark with RMSE) and how to
show it live in the existing React covariance-ellipse viewer.
Prereqs: 00–02.
Why it matters here: "demo-first, accuracy" means the benchmark is the primary deliverable.

1. Why synthetic data

The repo has **no ground-truth datasets** — only synthetic scenarios inside tests (`dvl_correction/tests/test_ekf_quality.py`). Ground truth is what lets us measure accuracy, so we
generate it: a known trajectory → simulate sensors from it → estimate → compare to truth.

2. The benchmark: examples/gp_dropout_benchmark.py

Generate ground truth

A smooth 3-D trajectory (e.g. a gentle lawnmower or sinusoidal path) sampled at high rate; from it
compute true position, velocity (body & nav), and attitude.

Simulate sensors (reuse the test pattern)

- **IMU @ ~100 Hz:** specific force + angular rate implied by the trajectory + bias + noise (mirror
`_run_stationary_with_accel_bias` in `tests/test_ekf_quality.py`).
- **DVL @ ~1 Hz:** body-frame velocity + noise, then **corrupt**:
  - inject **outlier spikes** at a few timestamps (e.g. ×5 magnitude), and
  - one or more **dropout windows** (e.g. 10 s and 30 s) where no DVL is emitted.
- (Optional) magnetometer yaw, as in `tests/test_magnetometer.py`.

Run both configurations

Feed identical sensor streams to two `NavigationFusionPipelines`:

- **baseline:** `gp_velocity=None`.
- **GP:** `gp_velocity=GpDvlVelocityModel(...)` enabled.

Metrics & artifacts

- **Position RMSE** and **max error** vs truth, overall and **restricted to the dropout windows**
(where the GP should help most).
- Velocity RMSE; orientation error (the literature shows orientation also improves).
- Pipeline diagnostics: `gp_denoised`, `gp_bridge_updates`, `gp_outlier_rejections`, `dvl_gate_rejections`.
- A **matplotlib** figure: true path vs baseline vs GP (top-down), plus a covariance-vs-time panel
showing the GP's covariance growing through the dropout while position error stays bounded.
- Print a small table and save the plot to `examples/out/`.

Success criterion

GP-enabled **position RMSE during/after dropouts is measurably lower** than baseline, with no
regression on clean segments. Record the numbers in the PR.

3. Optional live demo: the existing frontend

The React app already renders a position **mean + covariance ellipse** and speaks a fixed WebSocket
contract — so a real GP posterior drops in with **no frontend code changes**.

The contract (already defined)

- `frontend/web/src/types.ts` and `frontend/server/src/main.rs`: server→client at ~20 Hz emits
`tick { t, mean:[x,y], cov:[[xx,xy],[xy,yy]] }` and `status { state }`; client→server sends
`set_path`, `play`, `pause`, `reset`.
- The covariance ellipse is rendered by `frontend/web/src/canvas/ellipse.ts` (`ellipseFromCov`).
- The current `server/` is an explicit **stub** (`struct Sim`) that grows covariance linearly — the
README states "anything that speaks the contract can replace `server/`."

examples/gp_demo_ws.py (Python bridge)

- Run the benchmark scenario through the GP-enabled pipeline; at ~20 Hz emit
`tick { t, mean:[x,y]=position_m[:2], cov = 2×2 block of NavigationOutput.covariance position block }`.
- Serve it over WebSocket (add a dev-only `websockets` dependency to the example, **not** to `dvl_correction` core) and point `frontend/web/src/ws.ts` `WS_URL` at it.
- Optionally honor `play`/`pause`/`reset`/`set_path`; minimally, stream the canned scenario.

What the demo shows

Toggle the GP on/off and watch the covariance ellipse: **without** the GP it balloons during a DVL
dropout (and the mean drifts); **with** the GP it stays tight and the mean tracks truth — a vivid, intuitive picture of the benefit.

4. Tests (dvl_correction/tests/test_gp_velocity.py)

- **Unit:** `predict` returns finite `(3,)` mean + SPD `(3,3)` cov; variance at `t = last+10s` > variance at `t = last+0.5s`; `is_outlier` flags a ×5 spike and passes a normal sample; passthrough
when `not ready`.
- **Integration:** a small pipeline with the GP enabled produces the same `NavigationOutput` shape as
baseline, never raises, and `gp_bridge_updates > 0` across a simulated gap.
- **Regression:** `python3 -m unittest discover -s dvl_correction/tests` stays green with the GP off

by default (existing tests untouched).

5. Reproduce

```
pip install -e ./dvl_correction[gp]
python examples/gp_dropout_benchmark.py            # prints RMSE table, writes plot
# optional live demo:
python examples/gp_demo_ws.py                      # serves ws://127.0.0.1:8080/ws
cd frontend/web && npm install && npm run dev      # open http://localhost:5173
```

Key takeaways

- Build a **synthetic ground-truth + dropout/outlier** scenario (reuse the test harness) and compare

position RMSE with/without the GP, focusing on dropout windows — the primary proof.

- The frontend needs **no changes**: emit the existing `tick {mean, cov}` contract from a small Python

WS bridge to visualize the covariance ellipse staying tight through a dropout.

- Lock it in with unit + integration tests; keep existing tests green (GP off by default).

Further reading

- 04 — Edge and SPDE roadmap